

Fig. 1: Program flow-chart

variables with names such as 'A', 'a', RGB, real_array and so on. Images are read into MATLAB environment using 'imread' function:

```
>> imread('filename')
```

Let's say filename is 'f'. In order to see the information of this image, you can type "who f" command against the prompt. MATLAB will output following details:

Name	Size	Bytes	Class
f	1024x1024	1048576	uint8 array
Grand total is 1048576 elements using 1048576 bytes			

Image types. MATLAB toolbox supports four types of images: Intensity images, binary images, indexed images and RGB images.

Most monochrome image processing operations are carried out using binary or intensity images.

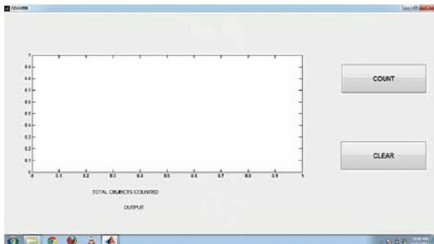


Fig. 2: Program output

Functions Used to Convert between Various Image Classes and Types

Name	Converts input to	Valid input image data classes
im2uint8	uint8	logical, uint8, uint16, and double
im2uint16	uint16	logical, uint8, uint16, and double
mat2gray	double (in range [0, 1])	double
im2double	double	logical, uint8, uint16, and double
im2bw	logical	uint8, uint16, and double

An intensity image is a data matrix whose values have been scaled to represent intensities.

Functions used in MATLAB to convert between various image classes and types are listed in the table.

Some main functions used in the code for object detection are:

rgb2gray. This function converts a true-colour RGB image into gray-scale intensity image:

```
I = rgb2gray(image)
```

regionprops. This function measures a set of properties for each connected component (object) in binary image BW:

```
STATS = regionprops(BW, properties)
```

Image 'BW' is a logical array that can have any dimension.

im2bw. This function is used to convert an intensity image into binary image types:

```
>> G=im2bw(f,T)
```

The `plotter.m` program uses all the above-mentioned functions. Flow-chart of the program is shown in Fig. 1.

Testing and operation

This project basically counts the number of objects of different sizes in an image file such as jpg, png and bmp. To test the program, you

can create an image file, say, `efy.jpg`, with square, round, rectangular, star and arrow objects and save in a folder, say, `C:\efytest`. Now define the path `C:\efytest\efy.jpg` in the code. Make sure that the images/objects are white with black background.

When you run the program, you can see control buttons 'count' and 'clear' as shown in the program output screenshot (Fig. 2). To count the graphics objects, just click 'count.' The program will automatically import the image file from the location already defined in the code. Even small-size components can be counted by reducing the value '2500' in the code below:

```
if (detectedobjects(j).Area)>2500;
```

Install MATLAB 7.10 or higher version in your system. Launch the application from your desktop. You will see a blank 'Command Windows' screen. Now proceed as follows:

1. Open the `plotter.m` file from 'File' menu.
2. Click 'Run' button to run the program. If the image file could not be loaded, check the location of the file